TEK-UP — ING-3 | Programmation Python

Corrigés Complets

TP5 · TP6 · Examen Session Principale 2025

Ce document regroupe les énoncés et corrections des TP5, TP6 et de l'examen de programmation Python (ING-3). Les codes sont présentés avec indentation Spyder (4 espaces).

TP 5 — Gestion des SMS (Fichiers & Dictionnaires)

Objectif : parser un fichier SMS → dictionnaire, filtrer spam/pub, recherche par mot.

Structure ligne : **NUMERO(10) DATE(10) HEURE(5) CONTENU**

1. Lire un fichier SMS → dictionnaire

✔ **Correction — lire_sms(nom_fichier)**

```
# -*- coding: utf-8 -*-
# sms.py
import re
import sys

def lire_sms(nom_fichier):
"""
Lit un fichier SMS et retourne un dictionnaire.
Cle = numero (str 10 chiffres)
Valeur = liste de messages [date, heure, contenu]
"""
sms_dict = {}
with open(nom_fichier, 'r', encoding='utf-8') as f:
for ligne in f:
ligne = ligne.strip()
if not ligne:
continue
numero = ligne[0:10] # 10 premiers caracteres
date = ligne[11:21] # JJ/MM/AAAA
heure = ligne[22:27] # HHhMM
contenu = ligne[28:].strip() # reste = contenu
message = [date, heure, contenu]
if numero in sms_dict:
sms_dict[numero].append(message)
else:
sms_dict[numero] = [message]
return sms_dict
```

■ *Indices fixes : numero=0:10, date=11:21, heure=22:27, contenu=28:*

2. Supprimer les SMS SPAM (numéro commençant par 0899)

✔ **Correction — est_spam() + sans_spam()**

```
def est_spam(contenu):
"""
Retourne True si le contenu contient un numero de telephone
commençant par 0899 (10 chiffres).
"""
return bool(re.search(r'\b0899\d{6}\b', contenu))

def sans_spam(sms_dict):
```

```
"""Retourne un nouveau dictionnaire sans les messages SPAM."""
resultat = {}
for numero, messages in sms_dict.items():
messages_propres = [m for m in messages
if not est_spam(m[2])]
if messages_propres:
resultat[numero] = messages_propres
return resultat
```

3. Supprimer les SMS publicitaires ($ ou >50% majuscules)

✔ Correction — est_pub() + sans_pub()

```
def est_pub(contenu):
"""
Retourne True si :
- contient '$', ou
- plus de la moitie des lettres sont des majuscules.
"""
if '$' in contenu:
return True
lettres = [c for c in contenu if c.isalpha()]
if lettres:
nb_maj = sum(1 for c in lettres if c.isupper())
if nb_maj / len(lettres) > 0.5:
return True
return False

def sans_pub(sms_dict):
"""Retourne un nouveau dictionnaire sans les SMS publicitaires."""
resultat = {}
for numero, messages in sms_dict.items():
messages_propres = [m for m in messages
if not est_pub(m[2])]
if messages_propres:
resultat[numero] = messages_propres
return resultat
```

■ *On calcule le ratio uniquement sur les lettres (isalpha()), pas les chiffres/espaces.*

4. Extraire la liste noire (pub OU spam)

✔ Correction — liste_noire()

```
def liste_noire(sms_dict):
"""
Retourne la liste des numeros ayant envoye
au moins un SMS spam OU publicitaire.
"""
blacklist = []
for numero, messages in sms_dict.items():
for m in messages:
if est_spam(m[2]) or est_pub(m[2]):
if numero not in blacklist:
blacklist.append(numero)
break
return blacklist
```

5. Chercher un mot exact dans les SMS

✔ Correction — chercher_mot()

```
def chercher_mot(sms_dict, mot):
"""
Retourne la liste des numeros dont un message contient
le mot EXACT. Ex : 'la' ne trouve PAS 'Carla'.
"""
```

```
resultat = []
for numero, messages in sms_dict.items():
for m in messages:
mots_message = m[2].split() # decoupage par espaces
if mot in mots_message:
if numero not in resultat:
resultat.append(numero)
break
return resultat
```

■ *split() sans arg → mots exacts. 'Carla'.split() = ['Carla'], donc 'la' ne matche pas.*

6. Bloc principal — afficher les SMS non publicitaires

✔ Correction — if __name__ == '__main__'

```
if __name__ == '__main__':
if len(sys.argv) != 2:
print("Usage : python sms.py <fichier_sms.txt>")
sys.exit(1)

nom_fichier = sys.argv[1]
sms = lire_sms(nom_fichier)
sms_propres = sans_pub(sms)

print("=== SMS non publicitaires ===")
for numero, messages in sms_propres.items():
for m in messages:
print(numero, m[0], m[1], m[2])
```

TP 6 — Programmation Orientée Objet

Exercice 1 — Classe Personne & Etudiant (Examen 2023)

Correction fournie par l'étudiant, mise en forme Spyder avec améliorations mineures.

Classes Personne et Etudiant complètes

✔ Correction — Personne + Etudiant + Programme principal

```
# -*- coding: utf-8 -*-
"""
Created on Fri May 8 10:12:36 2026
@author: ajmio

TP6 - Exercice 1 : Classe Personne & Etudiant
"""

# ============================================================
# Classe Personne
# ============================================================

class Personne:
def __init__(self, cin, nom, pre, adr, tel):
self.cin = cin
self.nom = nom
self.pre = pre
self.adr = adr
self.tel = tel

def Affiche(self):
print("NCIN :", self.cin)
print("Nom & Prénom :", self.nom, self.pre)
print("Contact :", self.adr, " - Tél :", self.tel)
```

```
# ============================================================
# Classe Etudiant (hérite de Personne)
# ============================================================

class Etudiant(Personne):
def __init__(self, cin, nom, pre, adr, tel, numcarte):
super().__init__(cin, nom, pre, adr, tel)
self.numcart = numcarte
self.matieres = ("Math", "Python", "BD", "Linux", "Ang")
self.coeff = (3, 2, 1, 2, 1)
self.__Notes = {} # attribut privé

def AjoutNote(self):
# --- Validation du code matière ---
while True:
matiere = input("Donner le code matière : ")
if matiere in self.matieres:
break
print("Code matière invalide.",
"Matières disponibles :", self.matieres)

# --- Validation de la note ---
while True:
try:
note = float(input("Donner la note (0-20) : "))
if 0 <= note <= 20:
break
else:
print("Note hors intervalle [0, 20].")
```

```
except ValueError:
print("Note invalide ! Veuillez saisir un nombre.")

self.__Notes[matiere] = note

def calcul_moy(self):
total = 0
somme_coeff = 0
for mat, note in self.__Notes.items():
ind = self.matieres.index(mat)
total += note * self.coeff[ind]
somme_coeff += self.coeff[ind]
if somme_coeff == 0:
return 0.0
return round(total / somme_coeff, 2)
```

```
# ============================================================
# Programme principal
# ============================================================

e = Etudiant(124, 'Ajmi', 'Ons', 'Tunis', 26001016, 5145545454)

print("=== Informations de l'étudiant ===")
e.Affiche()

print("\n=== Saisie des notes ===")
nb = int(input("Donner le nombre de notes : "))
for i in range(nb):
print(f"\n--- Note {i + 1} ---")
e.AjoutNote()

print("\nLa moyenne est :", e.calcul_moy())
```

■ *Ameliorations vs original : except ValueError (plus precis), garde-fou somme_coeff==0, round(,2) sur la moyenne.*

Exercice 2 — Gestion d'une Bibliothèque (Examen 2022)

1. Classe Exemplaire

✔ Correction — class Exemplaire

```
class Exemplaire:
def __init__(self, num, dispo=True):
self.num = num
self.dispo = dispo # True = disponible

def aff(self):
etat = 'Disponible' if self.dispo else 'Non disponible'
print("N°Exemplaire :", self.num)
print("Etat :", etat)

# Test
# e = Exemplaire('AZ789EE-1', True)
# e.aff()
# N°Exemplaire : AZ789EE-1
# Etat : Disponible
```

2. Classe Livre — toutes méthodes

✔ Correction — class Livre

```
class Livre:
def __init__(self, isbn, titre, auteur):
self.isbn = isbn
self.titre = titre
```

```
self.auteur = auteur
self.nbex = 0 # nombre d'exemplaires (public)
self.__L = [] # liste privée des Exemplaire

def affLiv(self):
print("ISBN :", self.isbn,
"| Titre :", self.titre,
"| Auteur :", self.auteur)

def affEx(self):
for ex in self.__L:
ex.aff()

def ajouterEx(self):
self.nbex += 1
num_ex = f"{self.isbn}-{self.nbex}"
self.__L.append(Exemplaire(num_ex, True))

def emprunter(self):
for ex in self.__L:
if ex.dispo:
ex.dispo = False
return ex.num
print("Livre non disponible")
return None

def rendre(self, num):
for ex in self.__L:
if ex.num == num:
ex.dispo = True
return
print("Exemplaire", num, "introuvable.")
```

3. Programme principal — Bibliothèque

✔ Correction — main

```
if __name__ == '__main__':
n = int(input("Nombre de livres a gérer : "))
bibliotheque = {} # dict[str, Livre]

for _ in range(n):
isbn = input("ISBN : ")
titre = input("Titre : ")
auteur = input("Auteur : ")
nbex = int(input("Nombre d'exemplaires : "))
livre = Livre(isbn, titre, auteur)
for _ in range(nbex):
livre.ajouterEx()
bibliotheque[titre] = livre

titre_emp = input("Titre du livre a emprunter : ")
if titre_emp in bibliotheque:
num = bibliotheque[titre_emp].emprunter()
if num:
print("Exemplaire emprunté :", num)
bibliotheque[titre_emp].affEx()
else:
print("Ce livre n'existe pas")
```

Examen Session Principale 2025

Exercice 1 (5 pts) — Tracé d'exécution des 6 codes

Code 1 — Lecture fichier (abc / def / ghi)

✔ Correction — Sortie attendue

```
file = open('abc.txt')
a = [i for i in file]
print(a)

# Sortie :
# ['abc\n', 'def\n', 'ghi']
#
# Explication : chaque ligne est lue avec son '\n' final.
# La derniere ligne n'a pas de '\n' si le fichier ne se termine
# pas par un saut de ligne.
```

■ *open() sans mode = 'r' par défaut. La list comprehension crée une liste de chaînes (lignes entières).*

Code 2 — split() + join()

✔ Correction — Sortie attendue

```
a = "ab cd"
print('*'.join(a.split()))

# a.split() --> ['ab', 'cd']
# '*'.join([...]) --> 'ab*cd'
#
# Sortie : ab*cd
```

Code 3 — Tentative de modification d'une chaîne (immuable)

✔ Correction — Sortie attendue

```
a = "abcdef"
a[1] = 'x'
print(a)

# Sortie : TypeError: 'str' object does not support item assignment
#
# Explication : les chaînes de caractères (str) sont IMMUABLES
# en Python. On ne peut pas modifier un caractère directement.
# Pour modifier, il faut créer une nouvelle chaîne :
# a = a[:1] + 'x' + a[2:] --> 'axcdef'
```

■ *Contrairement aux listes (mutables), les str sont immuables → TypeError à l'exécution.*

Code 4 — Double indexation négative

✔ Correction — Sortie attendue

```
mois = ['Mai', 'Juin', 'Juillet']
print(mois[-2][-2])

# mois[-2] --> 'Juin' (avant-dernier element de la liste)
# 'Juin'[-2] --> 'i' (avant-dernier caractere de 'Juin')
#
# Sortie : i
```

Code 5 — List comprehension avec expression conditionnelle

✔ Correction — Sortie attendue

```
a = [1,2,3,4,5,6,7,8,9]
b = [i if i%3==0 else 0 for i in a]
print(b)

# Pour chaque i :
```

```
# si i % 3 == 0 --> on garde i
# sinon --> on met 0
#
# i=1 -> 0 | i=2 -> 0 | i=3 -> 3
# i=4 -> 0 | i=5 -> 0 | i=6 -> 6
# i=7 -> 0 | i=8 -> 0 | i=9 -> 9
#
# Sortie : [0, 0, 3, 0, 0, 6, 0, 0, 9]
```

Code 6 — Lambda avec slicing et upper()

✔ Correction — Sortie attendue

```
print((lambda x: x[::-1].upper())("bonjour"))

# x[::-1] --> inverse la chaine --> 'ruojnob'
# .upper() --> tout en majuscules --> 'RUOJNOB'
#
# Sortie : RUOJNOB
```

Exercice 2 (5 pts) — Gestion des Salariés

Structure : BD_salaries = liste de dict{ 'nom', 'email', 'projets': [(nom, heures, taux), ...] }

1. ajouter_salarie()

✔ Correction — ajouter_salarie(BD_salaries)

```
BD_salaries = []

def ajouter_salarie(BD_salaries):
"""
Saisit les informations d'un salarié avec contrôles
et l'ajoute a BD_salaries.
"""
nom = input("Nom du salarié : ").strip()
while not nom:
print("Le nom ne peut pas être vide.")
nom = input("Nom du salarié : ").strip()

email = input("Email : ").strip()
while '@' not in email or '.' not in email:
print("Email invalide.")
email = input("Email : ").strip()

projets = []
nb_projets = int(input("Nombre de projets : "))
for i in range(nb_projets):
nom_p = input(f" Projet {i+1} — Nom : ").strip()
heures = float(input(f" Projet {i+1} — Heures : "))
taux = float(input(f" Projet {i+1} — Taux horaire : "))
projets.append((nom_p, heures, taux))

salarie = {'nom': nom, 'email': email, 'projets': projets}
BD_salaries.append(salarie)
print("Salarié", nom, "ajouté avec succès.")
```

2. salaire_total()

✔ Correction — salaire_total(BD_salaries, email)

```
def salaire_total(BD_salaries, email):
"""
Retourne le salaire total du salarié identifié par email.
Salaire = somme(heures * taux) sur tous ses projets.
"""
```

```
for salarie in BD_salaries:
if salarie['email'] == email:
total = sum(h * t for (_, h, t) in salarie['projets'])
return round(total, 2)
print("Salarié introuvable pour l'email :", email)
return None
```

3. classement_salaries()

✔ Correction — classement_salaries(BD_salaries, nom_projet)

```
def classement_salaries(BD_salaries, nom_projet):
"""
Trie les salariés par ordre décroissant des heures
travaillées sur le projet nom_projet.
"""
classement = []
for salarie in BD_salaries:
for (nom_p, heures, _) in salarie['projets']:
if nom_p == nom_projet:
classement.append((salarie, heures))
break

# Tri décroissant sur les heures
classement.sort(key=lambda x: x[1], reverse=True)

print("Classement pour le projet :", nom_projet)
for rang, (s, h) in enumerate(classement, 1):
print(f" {rang}. {s['nom']} — {h}h")
return classement
```

Exercice 3 (10 pts) — Gestion d'un Parking

1a & 1b. Classe Voiture — constructeur + afficher()

✔ Correction — class Voiture

```
class Voiture:
def __init__(self):
"""
Constructeur : demande la saisie de l'immatriculation.
Par défaut la voiture est non garée.
"""
self.immatriculation = input("Saisir l'immatriculation : ").strip()
self.est_garee = False

def afficher(self):
etat = 'Garée' if self.est_garee else 'Non garée'
print(" Immatriculation :", self.immatriculation)
print(" État :", etat)

# Exemple :
# v = Voiture() --> saisie : '123-TN-456'
# v.afficher()
# Immatriculation : 123-TN-456
# État : Non garée
```

■ *Le constructeur ne prend aucun paramètre : la saisie est interactive via input().*

2a–2e. Classe Parking complète

✔ Correction — class Parking

```
class Parking:
def __init__(self, nom, capacite):
"""Constructeur paramétré. voitures = liste privée vide."""
```

```
self.nom = nom
self.capacite = capacite
self.__voitures = [] # liste privée

def afficher_parking(self):
print("Parking :", self.nom,
"| Capacité :", self.capacite, "places")

def afficher_voitures(self):
if not self.__voitures:
print("Aucune voiture dans le parking.")
for v in self.__voitures:
v.afficher()

def ajouter_voiture(self, immatriculation):
"""
Cree un objet Voiture avec l'immatriculation donnée
et l'ajoute si le parking n'est pas plein.
"""
if len(self.__voitures) >= self.capacite:
print("Parking Plein")
return
# Instanciation sans appel de __init__ (qui lit input)
v = Voiture.__new__(Voiture)
v.immatriculation = immatriculation
v.est_garee = True
self.__voitures.append(v)
print("Voiture", immatriculation, "ajoutée.")

def sortir_voiture(self, immatriculation):
"""Marque la voiture comme non garée. Affiche erreur si absente."""
for v in self.__voitures:
if v.immatriculation == immatriculation:
v.est_garee = False
print("Voiture", immatriculation, "a quitté le parking.")
return
print("Voiture introuvable")
```

3. Programme principal — Parking

✔ Correction — main

```
if __name__ == '__main__':

n = int(input("Combien de parkings souhaitez-vous gérer ? "))
parkings = {} # dict[str, Parking]

for _ in range(n):
nom_p = input("Nom du parking : ").strip()
capa = int(input("Capacité : "))
parkings[nom_p] = Parking(nom_p, capa)

# Ajout d'une voiture
nom_p = input("Nom du parking pour l'ajout : ").strip()
if nom_p in parkings:
immat = input("Immatriculation de la voiture : ").strip()
parkings[nom_p].ajouter_voiture(immat)
print("--- Voitures dans le parking ---")
parkings[nom_p].afficher_voitures()
else:
print("Ce parking n'existe pas.")
```